

Important Concepts:

These four terms trip up almost everyone at first because they sound similar but have very different rules. Let's go one at a time.

SUBSTRING

Definition: a contiguous (unbroken) sequence of characters from a string.

Analogy: think of a string as a chain of connected beads. A substring is any unbroken segment you can cut out of that chain, you can't skip beads in the middle.

Example: for string "abcd" valid substrings:

"a", "ab", "abc", "abcd", "b", "bc", "bcd", "c", "cd", "d"

NOT a substring: "ac" (skips b, breaks contiguity)

Count of substrings for a string of length $n = n*(n+1)/2$

```
public class SubstringExample {  
    public static void printSubstrings(String s) {  
        int n = s.length();  
        for (int i = 0; i < n; i++) {  
            for (int j = i + 1; j <= n; j++) {  
                System.out.println(s.substring(i, j));  
            }  
        }  
    }  
}
```

```
=====
```

```
=====
```

```
=====
```

SUBSEQUENCE

Definition: a sequence derived from the original string/array by deleting zero or more elements WITHOUT changing the relative order of the remaining elements. Elements do NOT need to be contiguous.

Analogy: imagine picking some beads off the chain and stringing them back together, you can skip beads, but you can't reorder them.

Example: for string "abcd" valid subsequences: "a", "ac", "abd", "bd", "abcd", "" (empty) NOT a subsequence: "ba" (order changed, b comes before a in original)

Count of subsequences for a string of length $n = 2^n$ (including empty)

```
public class SubsequenceExample {
    public static void findSubsequences(String s, int index, String current) {
        if (index == s.length()) {
            System.out.println(current.isEmpty() ? "(empty)" : current);
            return;
        }
        // choice 1: skip current character
        findSubsequences(s, index + 1, current);
        // choice 2: include current character
        findSubsequences(s, index + 1, current + s.charAt(index));
    }
}
```

SUBSET

Definition: this is really the same concept as subsequence, but applied to arrays/sets rather than strings, usually when order doesn't matter at all (unlike subsequence where relative order is preserved). A subset is any combination of elements chosen from the original set, including the empty set and the full set.

Analogy: like choosing toppings for a pizza, you can pick any combination, and the "order" you picked them in doesn't matter, cheese+mushroom is the same subset as mushroom+cheese.

Example: for array [1, 2, 3] all subsets:

[], [1], [2], [3], [1,2], [1,3], [2,3], [1,2,3]

Count of subsets for an array of size $n = 2^n$

```
public class SubsetExample {  
    public static void findSubsets(int[] arr, int index, List<Integer> current) {  
        if (index == arr.length) {  
            System.out.println(current);  
            return;  
        }  
        // choice 1: don't include arr[index]  
        findSubsets(arr, index + 1, current);  
        // choice 2: include arr[index]  
        current.add(arr[index]);  
        findSubsets(arr, index + 1, current);  
        current.remove(current.size() - 1); // backtrack  
    }  
}
```

Quick distinction: subsequence usually refers to strings/sequences where order is preserved from the original. Subset usually refers to sets/arrays where order doesn't matter (a subset and its "reordering" are considered identical). In practice the counting logic and recursive tree are nearly identical, that's why they get grouped together.

SUBARRAY

I'd bet this comes up right after these three, so worth covering now. A subarray is the array equivalent of a substring, contiguous elements only.

Example: for array [1, 2, 3, 4] valid subarrays: [1], [1,2], [1,2,3], [1,2,3,4], [2], [2,3], etc. NOT a subarray: [1,3] (skips 2, breaks contiguity)

OTHER RELATED THINGS YOU SHOULD KNOW

Permutation: all possible orderings of a full set of elements, where order matters. For [1,2,3], count = $n! = 6$. Example orderings: [1,2,3], [1,3,2], [2,1,3], etc.

Combination: choosing k elements out of n where order does NOT matter. This is basically "subsets of a fixed size k ." Formula: $nCk = n! / (k! * (n-k)!)$

Why this matters for your interview prep specifically:

- **Sliding window pattern operates on substrings/subarrays (contiguous) — things like "longest substring without repeating characters"**
- **Backtracking pattern generates subsets, subsequences, permutations, combinations — things like "generate all subsets," "N-Queens," "combination sum"**
- **Dynamic programming often works on subsequences — things like "longest common subsequence," "longest increasing subsequence"**

So recognizing which of these four terms a problem is using tells you almost immediately which pattern family to reach for. "Substring" or "subarray" in the problem statement -> think sliding window or two pointers. "Subsequence" or "subset" -> think backtracking or DP.

Does this distinction make sense so far, or do you want me to trace through a small backtracking example (like generating all subsets of $[1,2,3]$) step by step the way we did with the sorting algorithms?